

Non-Invasive Diabetes Risk Stratification Using Rough Sets, Neural Networks and Fuzzy Inference

A hybrid soft-computing pipeline on the CDC Diabetes Health Indicators dataset

Dataset: UCI ML Repository #891 — CDC Diabetes Health Indicators
253,680 instances • 21 features • BRFSS 2015

Team: 4 members • **Scope:** Full semester

Target output: Working system + research paper + presentation

Complete workflow specification, methodology rationale,
compute allocation and execution plan

How to Use This Document

This is a **design specification**, not a tutorial and not a final report. It exists so that four people working on four different machines can build one coherent system without discovering in week nine that their pieces do not fit together.

Every technical term is explained the first time it appears, in a green **Plain English** box. You do not need prior knowledge of rough set theory, fuzzy logic, or neural networks to read this. You do need to read it in order, because later sections depend on decisions made in earlier ones.

The four box types

PLAIN ENGLISH

Explains a technical term or concept from scratch. Skip these if you already know the material.

TRAP

A specific way this project can fail. These are the highest-value parts of the document. Most of them are things that will not produce an error message — the code will run fine and give you a wrong answer that looks right.

WHY WE DIFFER FROM THE STANDARD APPROACH

Marks a place where we deliberately deviate from what the majority of published work on this dataset does, and explains the reason. These are collectively the argument for why our project is worth writing up.

COMPUTE

Which machine runs this, roughly how long it takes, and what the memory constraint is.

Ownership tags

Sections are tagged with who owns them:

M1 Rough Sets **M2 ANN** **M3 Fuzzy** **M4 Optim/Integration** **Shared**

Contents

- I **Framing: what this project actually claims** — why the obvious framing fails review, and the three fixes
- II **The data** — feature dictionary in plain language, known defects, what is measurable and what is not
- III **The pipeline** — six stages, each with theory, method, rationale and code sketch
- IV **Differentiation table** — standard approach vs. ours, side by side
- V **Experimental protocol** — the exact result tables you will fill in
- VI **Compute plan** — measured runtimes, the threading result, and why Machine B is optional
- VII **Software stack and environment** — libraries, versions, the CUDA trap
- VIII **Timeline** — ten weeks, with a compressed six-week fallback
- IX **Risk register** — what goes wrong and what you do about it
- X **Paper and presentation plan** — section outline, figure list
- A **Environment setup** — venv, CPU PyTorch, the interpreter trap, the threading benchmark
- B **First 48 hours checklist**
- C **Decision log** — the numbers to record as you go
- D **Glossary**

Part I — Framing

Owner: **Shared** — but this section determines what every other member builds, so read it before writing any code.

1.1 The problem with the obvious framing

The natural title for this project is "Early Diabetes Risk Detection." It is also the title that will get the project marked down, and here is exactly why.

The CDC Diabetes Health Indicators dataset is derived from the **Behavioral Risk Factor Surveillance System (BRFSS)**, an annual telephone health survey run in the United States. Three properties of that origin matter enormously:

- **It is cross-sectional, not longitudinal.** Every row is one person at one moment in time. Nobody is followed up. There is no "before" and "after."
- **The label is self-reported diagnosis, not a measurement.** The target variable answers the question "has a doctor ever told you that you have diabetes?" It is not a blood test result. There is no HbA1c value, no fasting plasma glucose, no oral glucose tolerance test anywhere in this data.
- **Every feature is measured at the same instant as the label.** Nothing in the data precedes the diagnosis in time.

"Early detection" means identifying disease before it becomes clinically apparent — which requires either a longitudinal design (predict who develops diabetes in the next five years) or a gold-standard biochemical label that catches people who have not yet been diagnosed. This dataset supports neither. A reviewer, an examiner, or a well-read classmate can dismantle an "early detection" claim in a single sentence.

FIX 1 — REFRAME THE CLAIM

We are not building an early-detection system. We are building a **non-invasive risk stratification and screening triage tool**: given only information a person can supply by answering questions — no needles, no lab, no clinic visit — how confidently can we sort them into risk tiers that determine who should be prioritised for an actual blood test?

This framing is (a) honest about what the data supports, (b) still genuinely useful and futuristic, since it targets low-resource and remote settings where lab access is the binding constraint, and (c) unattackable on the grounds above. It costs you nothing rhetorically and buys you complete defensibility.

1.2 The label is systematically wrong, and that is your best result

Consider what a label of 0 means in this dataset. It means "no doctor has ever told this person they have diabetes." That covers two very different groups:

- People who genuinely do not have diabetes.
- People who **do** have diabetes but have never been diagnosed — because they have not seen a doctor, cannot afford one, or have no insurance.

The CDC's own estimates put the undiagnosed fraction at roughly a fifth to a quarter of all diabetes cases in the US population. So a meaningful slice of your negative class is mislabelled.

This is not merely noise. It is **structured** noise. The probability of being undiagnosed is not random — it correlates with exactly the socioeconomic variables sitting in your feature matrix: `AnyHealthcare` (does this person have any health coverage), `NoDocbcCost` (did they skip seeing a doctor in the past year because of cost), and `Income`.

PLAIN ENGLISH — WHY STRUCTURED NOISE IS WORSE THAN RANDOM NOISE

If labels were flipped at random, a model would just be slightly less accurate everywhere. That is annoying but harmless. Here, the flipping is concentrated in poor and uninsured people. So the model learns, correctly given its training signal, that being poor and uninsured predicts *not having diabetes*. That is backwards from the epidemiological reality, and it means the model will systematically under-flag exactly the population that most needs flagging.

FIX 2 — TURN THE FLAW INTO A CONTRIBUTION

Almost no published work on this dataset says any of the above. Rather than mentioning it as a limitation in the last paragraph, we make it a measured result:

- Report model performance **broken down by** `AnyHealthcare`, `NoDocbcCost` and `Income` band. Quantify the gap.
- Do a sensitivity analysis: simulate correcting the label bias by re-weighting or by flipping a plausible fraction of high-risk-profile negatives in low-access subgroups, and show how conclusions move.
- Frame the finding as an equity result: a screening tool that under-detects in low-access populations inverts its own purpose.

This single thread does more to make the project publication-shaped than any amount of extra model tuning.

1.3 Two leakage traps

PLAIN ENGLISH — DATA LEAKAGE

Leakage is when information that would not really be available at prediction time sneaks into the model's inputs, making test performance look far better than real-world performance. The two common shapes are *temporal leakage* (a feature is actually a consequence of the outcome, not a cause) and *partition leakage* (the same information appears in both the training and test sets).

Trap A — consequence features masquerading as predictors

Three features are suspicious: `GenHlth` (self-rated general health, 1–5), `DiffWalk` (serious difficulty walking or climbing stairs) and `PhysHlth` (number of poor physical health days in the last 30). Each of these can plausibly be a **downstream effect** of having diabetes and its complications (neuropathy, fatigue, comorbidity burden) rather than an antecedent risk factor. A model leaning on them is partly detecting "this person is already unwell," which is much easier than the actual task.

What we do: run the entire headline experiment twice — once with the full 21 features, once with a **leakage-free subset** that drops these three. Report both. The difference is itself a finding, and reporting it demonstrates methodological maturity.

Trap B — duplicate rows straddling the split

This file contains a substantial number of **exact duplicate rows** — identical values across all 21 features and the label. This is expected: with mostly binary and coarsely-bucketed ordinal variables and a quarter of a million respondents, collisions are inevitable. Expect a five-figure count; measure it yourself on day one.

TRAP — THE SILENT SCORE INFLATION

If you split into train and test *before* handling duplicates, some test rows are byte-identical to training rows. The model has effectively memorised those test examples. Your reported test metric is then partly a memorisation score, and it will be inflated. Nothing will error. The number will just be too good.

Check on day one: `df.duplicated().sum()` and `df.drop(columns=['Diabetes_binary']).duplicated().sum()`. The second is the more interesting one — it counts *feature-identical* rows, which includes contradictory pairs where two people with identical answers have different labels. That count is directly relevant to Stage 2, because those contradictions are what will destroy classical rough set analysis.

What we do: deduplicate before splitting, and report headline results on the deduplicated data. Additionally run the primary configuration on the non-deduplicated data and report the delta in an appendix, so that our numbers are comparable to prior work that did not deduplicate. That comparison is a small contribution on its own.

1.4 Use the imbalanced version

Several versions of this data circulate. There is a 50/50 class-balanced variant with roughly 70,000 rows, created by downsampling the majority class. It is popular because it makes accuracy figures look impressive.

Use the full 253,680-row imbalanced file as primary. The class imbalance — roughly 14% positive, 86% negative — is the clinical problem. Removing it removes the problem. Run the balanced variant once, purely as a comparability check against published numbers, and put it in an appendix.

1.5 The claim we will defend

Using only non-invasive, self-reportable indicators, a rough-set-selected feature subset of roughly one third the original dimensionality retains the great majority of full-feature discriminative performance, while enabling a fuzzy inference layer that converts an opaque probability into an interpretable, actionable risk tier — and we quantify how the dataset's structured label bias degrades this system precisely in the low-healthcare-access subgroups it is meant to serve.

Every clause in that sentence is something one of the four members produces evidence for. If a result comes out the other way — say, rough set selection does *worse* than mutual information filtering — that is still a reportable result, and honestly reporting it is worth more than a fudged win.

Part II — The Data

Owner: **Shared** — everyone must agree on this before splitting up.

2.1 Feature dictionary

All 21 features plus the target. "Type" matters because it determines how each variable is handled in discretisation (Stage 2) and fuzzification (Stage 4). **Binary** = two values. **Ordinal** = ordered categories where the spacing between levels is not necessarily equal. **Count** = an integer number of things.

Feature	Type	Range	Meaning in plain language
Diabetes_binary TARGET	Binary	0 / 1	1 = respondent reports having been told by a health professional they have diabetes or prediabetes. 0 = neither. This is the thing we predict.
HighBP	Binary	0 / 1	Ever told they have high blood pressure.
HighChol	Binary	0 / 1	Ever told they have high cholesterol.
CholCheck	Binary	0 / 1	Had a cholesterol check within the past five years. Note this is a <i>healthcare access</i> proxy as much as a health measure.
BMI	Count	~12–98	Body Mass Index, weight in kg divided by height in metres squared. The only genuinely continuous-ish variable in the set.
Smoker	Binary	0 / 1	Has smoked at least 100 cigarettes in their lifetime. Note: this is <i>ever</i> , not <i>currently</i> .
Stroke	Binary	0 / 1	Ever told they had a stroke.
HeartDiseaseorAttack	Binary	0 / 1	Coronary heart disease or myocardial infarction.
PhysActivity	Binary	0 / 1	Any physical activity in the past 30 days outside of their job.
Fruits	Binary	0 / 1	Consumes fruit at least once per day.
Veggies	Binary	0 / 1	Consumes vegetables at least once per day.
HvyAlcoholConsump	Binary	0 / 1	Heavy drinker: more than 14 drinks per week for men, more than 7 for women.
AnyHealthcare	Binary	0 / 1	Has any kind of health coverage. Key variable for the label-bias analysis.
NoDocbcCost	Binary	0 / 1	In the past 12 months, needed to see a doctor but could not because of cost. Key variable for the label-bias analysis.
GenHlth	Ordinal	1–5	Self-rated general health. 1 = excellent, 5 = poor. Leakage-suspect.
MentHlth	Count	0–30	Days of poor mental health in the past 30 days.
PhysHlth	Count	0–30	Days of poor physical health or injury in the past 30 days. Leakage-suspect.
DiffWalk	Binary	0 / 1	Serious difficulty walking or climbing stairs. Leakage-suspect.
Sex	Binary	0 / 1	0 = female, 1 = male.
Age	Ordinal	1–13	Five-year age bands. 1 = 18–24, 2 = 25–29, 3 = 30–34, ... 12 = 75–79, 13 = 80 and over. Already discretised for you.
Education	Ordinal	1–6	1 = never attended / kindergarten only, 2 = grades 1–8, 3 = grades 9–11, 4 = grade 12 or GED, 5 = some college (1–3 yrs), 6 = college graduate (4+ yrs).
Income	Ordinal	1–8	Annual household income bands, 1 = under \$10k rising to 8 = \$75k and above.

2.2 Structural properties you must verify on day one

Do not take any of these on trust, including from this document. Verify each and record the number in a shared file, because several downstream design decisions depend on the actual values.

Property	Expectation	How to check
Row count	253,680	<code>len(df)</code>
Missing values	None (UCI reports zero)	<code>df.isna().sum().sum()</code>
Class prevalence	≈13–14% positive	<code>df.Diabetes_binary.mean()</code>
Exact duplicate rows	Five figures	<code>df.duplicated().sum()</code>
Feature-identical rows (ignoring label)	Larger than above	<code>df.drop(columns=[TARGET]).duplicated().sum()</code>
Contradictory groups	Substantial — this number drives Stage 2	Group by all 21 features; count groups containing both labels. See code in §3.3.
BMI distribution	Right-skewed, implausible extremes present	<code>df.BMI.describe()</code> , plus a histogram
MentHlth / PhysHlth	Enormous spike at 0, secondary spike at 30	Value counts. This zero-inflation matters for binning.

TRAP — "NO MISSING VALUES" IS NOT THE SAME AS "NO MISSING DATA"

BRFSS has genuine non-response, refusals, and "don't know" answers. The version distributed here has already had those rows removed or recoded upstream. So the clean dataset you receive is a **survivor** of a filtering process you did not observe and cannot audit. Anyone who could not or would not answer is gone. That is another selection effect worth one sentence in your limitations section — and it compounds the access-related bias from §1.2, since non-response also correlates with socioeconomic position.

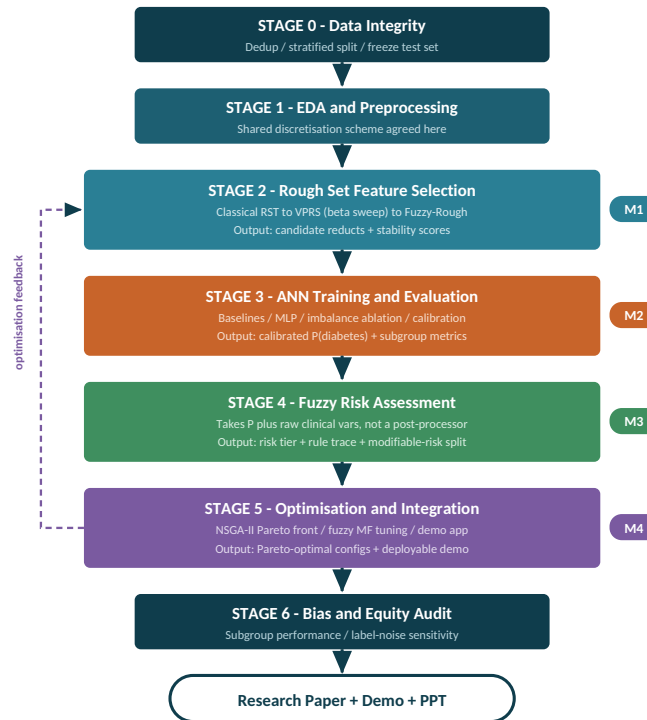
2.3 A note on prediabetes

A three-class version of this data exists (0 = none, 1 = prediabetes, 2 = diabetes). The binary version we use folds prediabetes into the positive class. This is defensible for a screening tool — both groups warrant a blood test — but it does mean the positive class is heterogeneous, mixing a mild metabolic state with established disease. Mention it explicitly. If time permits at the end, running the three-class version as a secondary experiment is a cheap way to add depth.

Part III — The Pipeline

Overview

Six stages. Stage 0 and 1 are shared groundwork. Stages 2–5 map onto the four members. The critical thing to notice is that **Stage 2 and Stage 3 must agree on discretisation**, and **Stage 3 and Stage 4 must agree on an output contract**. Those two interfaces are where four-person projects usually break.



3.1 Stage 0 — Data integrity Shared

This is a single afternoon of work and it protects everything downstream. Do it once, together, on one machine, and commit the outputs.

Steps

1. **Load** the 253,680-row file. Confirm shape, dtypes and the properties table in §2.2.
2. **Record** the count of exact duplicates, feature-identical rows, and contradictory groups. These numbers go in the paper.
3. **Deduplicate** exact duplicates. Keep the first occurrence.
4. **Split** into train / validation / test using stratification on the label, e.g. 70 / 15 / 15. Use a fixed random seed.
5. **Freeze the test set**. Save the row indices to a file, commit it, and do not look at the test set again until every modelling decision is final.
6. **Hash** the deduplicated dataframe and record the hash. Every member's script must assert this hash matches before running. This catches the situation where two people are unknowingly working on different data.

PLAIN ENGLISH — STRATIFIED SPLIT

A random split might, by chance, put a higher proportion of diabetic cases in the test set than the training set. *Stratified* splitting forces each split to have the same class proportion as the whole dataset. With a 14% minority class this matters; without it your metrics jitter between runs for reasons that have nothing to do with your model.

TRAP — THE FROZEN TEST SET IS NOT A SUGGESTION

The moment you evaluate on the test set, tweak something, and evaluate again, that test set has become a validation set and your final number is optimistic. Every decision — which reduct, which architecture, which threshold, which β — is made using the **validation** set. The test set is touched exactly once, at the end, to produce the numbers that go in the paper. Put this rule in your README and hold each other to it.

```

import pandas as pd, numpy as np, hashlib
from sklearn.model_selection import train_test_split

TARGET = "Diabetes_binary"
df = pd.read_csv("diabetes_binary_health_indicators_BRFSS2015.csv")

print("rows:", len(df))
print("exact dupes:", df.duplicated().sum())
print("feature-identical:", df.drop(columns=[TARGET]).duplicated().sum())
print("prevalence:", df[TARGET].mean().round(4))

# contradictory groups: same features, different label
g = df.groupby(list(df.columns.drop(TARGET)))[TARGET].nunique()
print("contradictory groups:", (g > 1).sum())

df = df.drop_duplicates().reset_index(drop=True)
h = hashlib.md5(pd.util.hash_pandas_object(df, index=False).values).hexdigest()
print("DATA HASH:", h) # every script asserts this

idx = np.arange(len(df))
tr, tmp = train_test_split(idx, test_size=0.30, stratify=df[TARGET], random_state=42)
va, te = train_test_split(tmp, test_size=0.50, stratify=df[TARGET].iloc[tmp], random_state=42)
np.savez("splits.npz", train=tr, val=va, test=te, data_hash=h) # COMMIT THIS FILE

```

COMPUTE

Laptop (Ultra 5 125H). Under one minute. The whole dataset is roughly 40–45 MB in memory as float64, and about half that as float32. Nothing here is remotely demanding.

3.2 Stage 1 — EDA and the shared discretisation scheme Shared, led by M1

Standard exploratory work — distributions, class-conditional rates, a correlation heatmap, mutual information ranking — but with one decision that is far more consequential than it looks.

The discretisation decision

PLAIN ENGLISH — DISCRETISATION

Discretisation means converting a numeric variable into a small number of labelled bins. BMI of 31.4 becomes "Obese." It is required by classical rough set theory, which can only reason about categories, and it is also what Stage 4's fuzzy system needs its linguistic labels to be built around.

Three variables need binning: **BMI** (about 80 distinct values), **MentHlth** and **PhysHlth** (0–30 each). Everything else is already binary or already an ordinal band.

WHY WE DIFFER

Most work on this dataset either skips discretisation entirely or uses equal-width binning, which is arbitrary and unjustifiable to a clinician. We use **clinically grounded cutpoints**, and we *benchmark* them against two automatic methods rather than just asserting they are better.

Proposed scheme

Variable	Bins	Justification
BMI	<18.5 Underweight • 18.5–24.9 Normal • 25–29.9 Overweight • 30–34.9 Obese I • ≥35 Obese II+	WHO standard categories. Instantly interpretable to any clinician, and reusable directly as fuzzy set cores in Stage 4.
MentHlth	0 • 1–13 • 14–30	Zero-inflation means 0 deserves its own bin. The 14-day cutoff is the conventional "frequent mental distress" threshold in BRFSS literature.
PhysHlth	0 • 1–13 • 14–30	Same logic, and it keeps the two symmetric.

Benchmark against: (a) equal-width binning into the same number of bins, and (b) a supervised method such as MDLP or ChiMerge, which chooses cutpoints to maximise information about the label. Report reduct size and downstream performance under all three. If the clinical cutpoints do about as well as the supervised ones, that is a nice result: interpretability is free here.

TRAP — FITTING THE DISCRETISER ON ALL THE DATA

Supervised discretisers such as MDLP *look at the labels* to choose cutpoints. If you fit one on the entire dataset and then split, label information from your test set has flowed into your preprocessing. Fit the discretiser on the training split only, then apply the learned cutpoints to validation and test. The same rule applies to scalers and to reduct computation.

EDA outputs to produce

- Class-conditional positive rate for every binary feature (a horizontal bar chart of $P(\text{diabetes} \mid \text{feature}=1)$ vs $P(\text{diabetes} \mid \text{feature}=0)$). This is your most readable single figure.
- BMI and Age distributions split by class.

- Spearman correlation heatmap (Spearman rather than Pearson, because almost everything is ordinal or binary).
- Mutual information ranking of all 21 features. This becomes a baseline feature-selection method to compare rough sets against.
- Prevalence by `Income`, `AnyHealthcare`, `NoDocbcCost` — the foundation of the Stage 6 equity analysis.

COMPUTE

Laptop. Five to ten minutes total. Use matplotlib and seaborn. Save every figure at 300 dpi directly into a `figures/` folder with meaningful filenames — you will want them for the paper and re-running the notebook to regenerate a plot in week nine is a waste of an evening.

3.3 Stage 2 — Rough Set Feature Selection M1

This is the section with the most conceptual content and the most ways to go wrong. Read all of it before writing code.

3.3.1 Rough set theory from zero

Rough Set Theory (RST) was introduced by Zdzisław Pawlak in 1982. Its central idea is disarmingly simple: **if two objects have identical values on the attributes you are looking at, you cannot tell them apart**. Everything else follows from that.

PLAIN ENGLISH — THE FIVE CORE TERMS

Information system / decision table. Just a table. Rows are objects (people). Columns are *condition attributes* (our 21 features) plus one *decision attribute* (the diabetes label).

Indiscernibility relation. Pick a subset B of the attributes. Two people are *indiscernible with respect to B* if they have exactly the same values on every attribute in B . This relation carves the population into groups, called **equivalence classes** or **granules**. In plain terms: a granule is "everyone who gave the same answers to this particular set of questions."

Lower approximation. Take the set of all diabetic people. The lower approximation is the union of all granules that are *entirely* diabetic. These are the people we can classify with certainty using only attributes B — everyone who answered like them also has diabetes.

Upper approximation. The union of all granules containing *at least one* diabetic person. These are the people who might be diabetic.

Boundary region. Upper minus lower. Granules containing a mix of diabetics and non-diabetics. Here, attributes B are insufficient to decide. If the boundary region is empty, the set is *crisp*; if not, it is *rough* — which is where the theory's name comes from.

A worked toy example

Six people, two attributes (HighBP, Smoker), one decision (Diabetes):

Person	HighBP	Smoker	Diabetes
u1	1	1	1
u2	1	1	1
u3	1	0	0
u4	0	0	0
u5	0	1	1
u6	0	1	0

The granules are $\{u1, u2\}$ at (1,1), $\{u3\}$ at (1,0), $\{u4\}$ at (0,0), and $\{u5, u6\}$ at (0,1).

- Lower approximation of "Diabetes = 1" is **$\{u1, u2\}$** — the only fully-diabetic granule.
- Upper approximation is **$\{u1, u2, u5, u6\}$** .
- Boundary region is **$\{u5, u6\}$** : two people who gave identical answers but have different outcomes. The attributes simply cannot separate them.

PLAIN ENGLISH — POSITIVE REGION, DEPENDENCY DEGREE, REDUCT, CORE

Positive region $POS_B(D)$ is the union of the lower approximations of every decision class — all the objects we can classify with certainty using attribute subset B . Here that is $\{u1, u2\} \cup \{u3, u4\}$ = four objects.

Dependency degree $\gamma_B(D) = |POS_B(D)| / |U|$, the fraction of the population classifiable with certainty. Here $\gamma = 4/6 \approx 0.667$. It ranges from 0 (attributes tell you nothing) to 1 (attributes fully determine the decision).

Reduct. A minimal subset of attributes that preserves γ at the same value as the full attribute set. "Minimal" is strict: remove any single attribute from a reduct and γ drops. This is the feature selection output.

Core. The intersection of all reducts — attributes that appear in every possible reduct and are therefore indispensable.

WHY ROUGH SETS AT ALL?

Unlike most feature selection methods, RST needs **no external parameter, no distributional assumption and no threshold** chosen by the user. It does not assume linearity like LASSO, does not require a probability model, and does not need you to pick "top k" as mutual information does. It derives the answer from the structure of the data alone. It also produces, as a by-product, **decision rules in IF-THEN form**, which feed directly into Stage 4's fuzzy rule base. That last property is why RST belongs in this pipeline rather than being an arbitrary choice of feature selector.

3.3.2 Why classical rough sets will fail on this data — in both directions

This is the single most important insight in the document, and it is what turns a routine application into a paper.

Failure mode A — without discretisation, granules become singletons

BMI has roughly 80 distinct values; MentHlth and PhysHlth have 31 each. Combined with the ordinal variables, nearly every one of the 250,000 rows becomes its own unique attribute vector. Every granule has size one. Every granule is trivially pure. Therefore $\gamma \approx 1.0$ for almost any attribute subset, greedy reduct search terminates after three or four attributes, and the "reduct" you get is meaningless — it is an artefact of high cardinality, not a discovery about diabetes.

Failure mode B — with discretisation, the boundary region swallows everything

Bin BMI into five categories and MentHlth/PhysHlth into three, and you now have roughly 21 low-cardinality attributes over a quarter of a million people. Enormous numbers of people share identical answer profiles — and among them, some have diabetes and some do not. Every such granule is contradictory and lands in the boundary region. The positive region collapses, γ drops toward zero, and classical reducts become degenerate again.

TRAP — THIS FAILURE IS SILENT

Neither failure mode raises an exception. In case A you get a suspiciously tiny reduct with $\gamma = 1.0$ and might celebrate. In case B you get γ near zero and might assume you have a bug. Neither is a bug. Both are the theory correctly telling you that Pawlak's crisp formulation does not fit noisy, high-volume survey data. **Measure γ explicitly under both regimes and put the numbers in the paper** — they are your motivation section.

3.3.3 The fix, part one: Variable Precision Rough Sets

Variable Precision Rough Sets (VPRS), due to Ziarko (1993), relaxes the strictness of the lower approximation. Instead of demanding that a granule be **100%** one class to count as certain, it demands only that the granule be at least β proportion one class, where β is a threshold you choose in $(0.5, 1]$.

PLAIN ENGLISH — WHAT B BUYS YOU

Suppose a granule contains 100 people, 92 diabetic and 8 not. Classical RST throws the whole granule into the boundary region as "uncertain" — eight exceptions destroy the information in the other ninety-two. VPRS with $\beta = 0.9$ says: 92% is close enough, treat this granule as certain. The eight are treated as noise, which for survey data is precisely the right attitude.

Applied to our toy example: the granule $\{u5, u6\}$ is 50% diabetic. With $\beta = 0.6$ it stays in the boundary. With $\beta = 0.5$ it is admitted, and $\gamma_{0.5} = 1.0$. So β directly controls how much noise tolerance the analysis has.

WHY WE DIFFER — THE B-SENSITIVITY CURVE

Papers that use VPRS typically pick one β value, often without justification. We instead sweep β from 0.50 to 1.00 in steps of 0.05 and plot, on a single figure: (i) reduct size, (ii) γ_β , and (iii) downstream ANN PR-AUC, all against β .

That one figure answers a question the literature on this dataset has not asked: how much noise tolerance does population survey data actually require before rough set analysis becomes informative? It is cheap to produce and it is the most likely single figure to survive into a publication.

The imbalance complication — and an opening for a genuine contribution

Here is a subtlety that will bite you if you do not anticipate it. Our positive class is only 14% of the population. That means the *typical* granule is overwhelmingly non-diabetic — a granule that is 88% negative is entirely unremarkable, because the base rate is 86%.

With a fixed β of, say, 0.85, almost every granule qualifies for the lower approximation of the **negative** class, while essentially none qualifies for the positive class. γ_β becomes large but is being driven almost entirely by the majority class. You have measured that most people do not have diabetes, which you already knew.

PROPOSED METHODOLOGICAL CONTRIBUTION — PREVALENCE-RELATIVE VPRS

Rather than an absolute threshold, define granule certainty **relative to the base rate**. A granule is admitted to the β -lower approximation of the positive class if

$$P(D = 1 \mid \text{granule}) / P(D = 1) \geq \lambda$$

i.e. if its positive rate exceeds the population prevalence by a lift factor λ . A granule that is 40% diabetic against a 14% base rate carries a lift of about 2.9 and is highly informative, even though 40% would fail any conventional absolute β .

Also report γ_β **per decision class** rather than pooled, so majority-class dominance is visible rather than hidden. This is a small, defensible, clearly-motivated extension to VPRS for imbalanced data, and "a modest methodological extension with an ablation showing it helps" is exactly the shape of a good undergraduate research contribution. Do not oversell it; do measure it against standard VPRS.

3.3.4 The fix, part two: Fuzzy-Rough Feature Selection

The second escape route removes the need to discretise at all. Fuzzy-Rough Feature Selection (FRFS) replaces the crisp indiscernibility relation with a **fuzzy similarity relation**: instead of two people being either identical or not on BMI, they are similar *to a degree* between 0 and 1, decaying as their BMI values diverge.

Lower and upper approximations become fuzzy, computed with t-norms and t-conorms (see §3.5.1 for what those are), and the dependency degree becomes a fuzzy positive region. BMI is handled natively at full resolution, and the arbitrary cutpoint problem disappears.

Its second virtue is conceptual: it puts fuzzy machinery into the feature selection stage, which makes the fuzzy inference system in Stage 4 a natural continuation of the same mathematical framework rather than a bolted-on afterthought. That coherence is worth real credit in a soft computing course.

TRAP — FRFS IS $O(N^2)$ AND $N = 250,000$

Naive FRFS computes a fuzzy similarity between every pair of objects. At 250,000 objects that is 62.5 billion pairs. As float32 that is roughly 250 gigabytes for a single similarity matrix. This will not run on either of your machines, or on any machine you have access to.

Mitigation: run FRFS on stratified subsamples of 8,000–10,000 objects. At 10,000 the matrix is 10^8 entries, about 400 MB in float32 — comfortable. Repeat over 10–20 independent subsamples and report the **selection frequency** of each attribute across runs. This converts a computational limitation into a stability analysis, which is more informative than a single deterministic run would have been.

3.3.5 The algorithm that actually scales: QuickReduct via groupby

The textbook algorithm for finding reducts builds a **discernibility matrix** — an $n \times n$ structure recording, for each pair of objects, which attributes distinguish them. At $n = 250,000$ that is 62.5 billion cells. Do not attempt it. Any tutorial that shows you a discernibility matrix is implicitly assuming a dataset with a few hundred rows.

Instead, note that the dependency degree only requires knowing, **for each distinct combination of attribute values, the label distribution within it**. That is exactly a `groupby`. It runs in roughly $O(n \log n)$, uses tens of megabytes, and completes in under a second per evaluation on your laptop.

```
import numpy as np, pandas as pd

def gamma_vprs(df, B, target, beta=1.0):
    """Dependency degree of attribute subset B, with VPRS threshold beta.
    beta = 1.0 recovers classical Pawlak rough sets."""
    if not B:
        return 0.0
    g = df.groupby(list(B))[target].agg(['size', 'sum'])
    p_pos = g['sum'] / g['size'] # positive rate within each granule
    purity = np.maximum(p_pos, 1 - p_pos) # majority-class proportion
    certain = g['size'][purity >= beta].sum() # objects in beta-positive region
    return certain / len(df)

def quickreduct(df, attrs, target, beta=1.0, tol=1e-6):
    """Greedy forward selection: repeatedly add the attribute that most
    increases gamma, until gamma stops improving."""
    full = gamma_vprs(df, attrs, target, beta)
    R, cur = [], 0.0
    while cur < full - tol:
        best, best_g = None, cur
        for a in attrs:
            if a in R: continue
            g = gamma_vprs(df, R + [a], target, beta)
            if g > best_g:
                best, best_g = a, g
        if best is None:
            break # no attribute improves gamma
        R.append(best); cur = best_g
    return R, cur, full
```

Cost accounting. QuickReduct over 21 attributes evaluates at most $21 + 20 + 19 + \dots \approx 231$ subsets. Each is one groupby over 250,000 rows, well under a second. So a full reduct search finishes in **two to four minutes**. An eleven-point β sweep is under an hour. This is entirely laptop work.

TRAP — GREEDY SEARCH FINDS A REDUCT, NOT THE REDUCT

QuickReduct is a greedy heuristic. It is not guaranteed to find the minimal reduct, and different attribute orderings can yield different answers. Do not write "the reduct" in your paper — write "a reduct obtained by greedy forward selection." Then, to strengthen the claim, hand the problem to Member 4: a genetic algorithm searching the space of attribute subsets can find smaller reducts than greedy search, and comparing GA-found reducts against QuickReduct reducts is a clean, self-contained experiment that connects Stage 2 to Stage 5.

3.3.6 Reduct stability

A reduct computed once on one training split is a single sample from a distribution. If resampling the data yields a completely different attribute set each time, the reduct is not telling you anything durable about diabetes — it is telling you about that particular sample.

Protocol: draw 30–50 bootstrap resamples of the training set. Compute a reduct on each. Report for every attribute its **selection frequency** across resamples, and summarise overall agreement with a set-similarity index (Jaccard or Kuncheva). An attribute selected in 48 of 50 resamples is a real finding; one selected 12 times out of 50 is noise.

WHY WE DIFFER

The overwhelming majority of applied rough set papers report a single reduct from a single run with no uncertainty quantification whatsoever. Adding stability analysis costs you one loop and roughly an hour of compute, and it is the difference between "we ran an algorithm" and "we characterised a method's behaviour." Reviewers notice this.

COMPUTE — STAGE 2 ALLOCATION

Task	Machine	Est. time
Discretisation, compressed table	A	< 1 min
Classical QuickReduct ($\beta=1$)	A	2–4 min
β sweep, 11 values	A	30–50 min
Prevalence-relative variant, λ sweep	A	30–50 min
Bootstrap stability, 50 resamples $\times$ 3 β	A, 4 workers	~30 min
FRFS, 15 subsamples of 6,000	A	~1 h

All of Stage 2 runs on Machine A. None of it touches a GPU — rough set computation is groupby and set operations, which are CPU and memory bound. The bootstrap loop is embarrassingly parallel; run it with `joblib.Parallel(n_jobs=4)` per Part VI.

The one memory constraint: FRFS builds a pairwise similarity matrix. At a 6,000-object subsample that is 36M float32 entries, roughly 145 MB — comfortable. Do not raise the subsample above about 8,000 on a 16 GB machine, and never attempt it on the full dataset (see the trap in §3.3.4).

3.3.7 Deliverables from Stage 2

1. A table of γ under: no discretisation, clinical discretisation, equal-width, supervised discretisation. Demonstrating both failure modes numerically.
2. The β -sensitivity figure (reduct size and γ_β vs β).
3. Five to eight candidate reducts, saved as JSON lists of column names, handed to Member 2.
4. An attribute stability table with selection frequencies.
5. A set of extracted IF–THEN decision rules from the best reduct, with support and confidence, handed to Member 3 as raw material for the fuzzy rule base.

INTERFACE CONTRACT WITH MEMBER 2

Reducts must be delivered as a JSON file mapping a short name to a list of exact column names, e.g. `{"vprs_b085": ["HighBP", "BMI", "Age", "GenHlth", ...]}`. Member 2's training script takes that name as a command-line argument. Agree this format in week two. If Member 2 has to hand-edit a list of features every time Member 1 produces a new reduct, you will lose days to it.

3.4 Stage 3 — ANN, Prediction and Evaluation M2

3.4.1 What the network is and what it is not

PLAIN ENGLISH — A MULTILAYER PERCEPTRON IN ONE PARAGRAPH

An MLP is a stack of layers. Each layer takes a vector of numbers, multiplies it by a matrix of learnable **weights**, adds a **bias**, and applies a non-linear **activation function** (we use ReLU, which simply replaces negatives with zero). Stacking these lets the network represent interactions between features that a linear model cannot. The final layer outputs a single number squashed by a **sigmoid** into the range 0–1, interpreted as the probability of diabetes. Training means: feed a batch of examples forward, measure error with a **loss function** (binary cross-entropy), compute how each weight contributed to that error via **backpropagation**, and nudge every weight slightly in the improving direction. One full pass over the training data is an **epoch**. Repeat for many epochs.

Two supporting concepts you will use immediately:

- **Overfitting** — the network memorises training data instead of learning generalisable patterns; training loss keeps falling while validation loss rises. Countered by **dropout** (randomly zeroing a fraction of neurons during training, forcing redundancy) and **early stopping** (halting when validation loss stops improving).
- **Batch size** — how many examples are processed before each weight update. Larger batches are faster per epoch and more stable; smaller batches generalise slightly better. For 250,000 rows, 512 or 1024 is a sensible default.

WHY AN ANN HERE, HONESTLY

You should expect gradient boosting to match or beat the MLP on this data. Tabular data with mostly binary features is boosting's home territory, and this is a well-documented pattern. **Say so in the paper.** The MLP's justification in this pipeline is not that it is state of the art — it is that (a) it produces smooth, differentiable probability estimates that the fuzzy layer can consume, (b) its architecture and weights can be jointly optimised by Member 4's evolutionary search, and (c) it is the neural component of a *soft computing* system, which is the course's subject. Claiming SOTA and being wrong is much worse than stating the real rationale and being right.

3.4.2 Metrics — and the one you must never lead with

TRAP — THE 86% ACCURACY THAT MEANS NOTHING

A model that predicts "no diabetes" for every single person achieves roughly 86% accuracy on this dataset, while identifying zero diabetics. It is completely useless and it beats many published accuracy figures. **Accuracy is not a valid headline metric on imbalanced data.** Compute it if you like, for comparability, but never lead with it and always print the majority-class baseline next to it.

PLAIN ENGLISH — THE METRICS THAT DO WORK

Precision = of the people we flagged as at-risk, what fraction actually are. **Recall** (sensitivity) = of the people who actually have diabetes, what fraction did we catch. These trade off: flag everyone and recall is perfect but precision equals the base rate.

PR-AUC (area under the precision–recall curve, also called average precision) summarises that trade-off across all thresholds. Its baseline is the class prevalence, ≈ 0.14 . This is our **primary metric** because it focuses entirely on the minority class.

ROC-AUC is the probability the model scores a random positive above a random negative. Report it for comparability with prior work, but treat it as secondary — it is optimistic under heavy imbalance because the huge negative class makes the false-positive rate look small.

Brier score is the mean squared error of the predicted probabilities. Lower is better. It measures **calibration**: whether, among people assigned a 30% risk, roughly 30% actually have diabetes.

WHY CALIBRATION IS NON-NEGOTIABLE IN THIS PIPELINE

In a standard classification paper, calibration is a nice-to-have. Here it is structural. Stage 4 feeds the predicted probability into fuzzy membership functions with linguistic labels like "Moderate risk." If the model is miscalibrated — if outputs of 0.4 actually correspond to a 12% true rate — then "Moderate" is a meaningless word and the entire fuzzy layer is built on sand. Report a **reliability diagram** (predicted probability bucket vs observed frequency) alongside the Brier score, and apply Platt scaling or isotonic regression on the validation set if calibration is poor.

Reporting set

Metric	Role	Notes
PR-AUC / Average Precision	Primary	Baseline = prevalence ≈ 0.14 . Always print the baseline next to it.
Recall at fixed precision	Primary	Pick an operating point a clinician would use, e.g. precision = 0.30, and report the recall achieved there.
Precision at fixed recall	Primary	e.g. at recall = 0.80. "To catch 80% of cases, how many people do we send for a blood test?"
Brier score + reliability diagram	Primary	Required for the fuzzy handoff, as above.
ROC-AUC	Secondary	For comparison against published work on this dataset.
Accuracy, F1	Reference only	Always beside the majority-class baseline.
Per-subgroup PR-AUC and recall	Key result	By Sex, Age band, Income, AnyHealthcare, NoDocbcCost. This is Stage 6.

3.4.3 Handling class imbalance

TRAP — DO NOT REACH FOR SMOTE REFLEXIVELY

SMOTE creates synthetic minority examples by interpolating between real ones. On this dataset, most features are binary. Interpolating halfway between HighBP = 0 and HighBP = 1 yields 0.5, which corresponds to no human being. Vanilla SMOTE here manufactures physically meaningless training examples, and it is applied constantly in published work on this dataset without comment.

If you oversample, use **SMOTE-NC**, the variant designed for mixed categorical/continuous data, which takes the majority category among neighbours rather than interpolating. And note that **all** oversampling destroys calibration by inflating the apparent base rate — which, per §3.4.2, matters a great deal here.

Run all four of these as a proper ablation, on the same feature set, with the same architecture:

1. **No resampling + threshold tuning.** Train normally, then choose the decision threshold on the validation set to hit your target operating point. Often the strongest and simplest option, and it preserves calibration.
2. **Class-weighted loss.** Weight the positive class by roughly the inverse of its frequency (`pos_weight` in PyTorch's `BCEWithLogitsLoss`). Cheap, effective, no synthetic data.
3. **SMOTE-NC oversampling.** The literature-standard comparison point.
4. **Random undersampling** of the majority class.

Report PR-AUC *and* Brier for each. The likely finding — that resampling gives little or no PR-AUC benefit while measurably degrading calibration — is a small, clean, honest contribution of its own.

TRAP — RESAMPLE INSIDE THE FOLD, NEVER OUTSIDE

Oversampling before splitting means synthetic points generated from a training example can land in your validation set. Catastrophic optimism, no error message. Resampling must happen strictly inside the training fold, after the split. Use an `imblearn.Pipeline` so this is structurally impossible rather than merely remembered.

3.4.4 Architecture and baselines

Do not over-engineer. On 21 or fewer tabular inputs, a small network is appropriate and a large one will simply overfit more slowly.

```
Input (|reduct| features, standardised)
  → Dense(64) + BatchNorm + ReLU + Dropout(0.3)
  → Dense(32) + BatchNorm + ReLU + Dropout(0.2)
  → Dense(1) + Sigmoid

Loss: BCEWithLogitsLoss(pos_weight = n_neg / n_pos)
Optimiser: AdamW, lr 1e-3, weight_decay 1e-4
Batch: 1024 | Epochs: up to 100 | EarlyStopping on val PR-AUC, patience 10
```

Search over roughly: hidden sizes {(32,16), (64,32), (128,64), (128,64,32)}, dropout {0.1, 0.2, 0.3, 0.4}, learning rate {3e-4, 1e-3, 3e-3}. Use Optuna with 40–80 trials rather than an exhaustive grid.

Mandatory baselines

Model	Why it is in the table
Majority-class predictor	The floor. Makes the accuracy trap visible to the reader.
Logistic regression	Interpretable, fast, and on this dataset surprisingly hard to beat. If your MLP does not beat it, that is a finding worth stating.
Decision tree / Random forest	Natural comparison for rough sets, which are also rule-based.
LightGBM or XGBoost	The realistic strong baseline for tabular data.
MLP	Our pipeline component, per the rationale above.

3.4.5 Statistical rigour

This is what separates a project from a paper.

- **Multiple seeds.** Every configuration trained with at least five random seeds. Report **mean ± standard deviation**, never a single run. A single run is not a measurement.
- **Cross-validation on the training portion.** Repeated stratified k-fold (5 folds × 3 repeats) for model selection. The frozen test set stays untouched.
- **Significance testing.** When you claim model A beats model B, back it with a test. McNemar's test for comparing two classifiers on the same test set; a paired 5×2 cv t-test for cross-validated comparisons. "0.812 vs 0.807" is not a result unless you show the difference exceeds run-to-run noise.
- **Preprocessing inside the fold.** Scaling, discretisation, resampling and reduct computation all fit on training data only, inside each fold.
- **Determinism.** Fix seeds for Python, NumPy and PyTorch; set `torch.use_deterministic_algorithms(True)` where feasible. Log the seed with every result row.

COMPUTE — STAGE 3 ALLOCATION

These figures are **measured on Machine A**, not estimated. See Part VI for the benchmark and Appendix A for the configuration that produces them.

Task	Machine	Time
LR, decision tree, forest baselines	A	Seconds
Single MLP train, 100 epochs	A, 4 threads	~55 s
LightGBM Optuna, 100 trials	A, 4 workers	20–40 min
Full grid: 8 feature sets $\times$ 4 imbalance strategies $\times$ 5 seeds = 160 runs	A, 4 workers	~75 min
Optuna hyperparameter search, 60 trials	A, 4 workers	~25 min

Nothing in Stage 3 is an overnight job. The entire modelling workload for this project is a few hours of wall-clock time on the laptop. Plan accordingly: you can afford to re-run a sweep after finding a bug, which means you should not be precious about getting each run right first time.

Why it is this fast: the dataset is ~20 MB in float32 and the model has roughly 3,500 parameters. Per-step time is dominated by Python and framework overhead, not arithmetic. Two consequences: (i) keep the whole dataset as one resident tensor and index into it, rather than using a `DataLoader` — on 20 MB the worker-process pickling costs more than the work itself; (ii) larger batches amortise loop overhead well here, so 2048 is worth testing against 1024.

3.4.6 The output contract with Member 3

INTERFACE CONTRACT — AGREE THIS IN WEEK TWO

Member 2 delivers, for every row in validation and test, a CSV containing: `row_id`, `p_calibrated`, the true label, and the raw values of the clinical variables Member 3's fuzzy system needs (`BMI`, `Age`, `HighBP`, `PhysActivity`, `Smoker`, `HighChol`). Member 3 builds against that file. This lets Stage 4 development start before the final model is chosen — Member 3 can begin with an early, imperfect model's outputs and swap in the final ones later without changing a line of code.

3.5 Stage 4 — Fuzzy Risk Assessment M3

3.5.1 Fuzzy logic from zero

PLAIN ENGLISH — FUZZY SETS AND THE VOCABULARY

Crisp set: an element either belongs or does not. "Obese" defined as $BMI \geq 30$ means 29.9 is not obese and 30.0 is, which is clinically absurd.

Fuzzy set: membership is a degree between 0 and 1. BMI 29.9 might be 0.85 "Overweight" and 0.15 "Obese" simultaneously. The function mapping a value to its degree is the **membership function** (MF), usually triangular or trapezoidal.

Linguistic variable: a variable whose values are words. "BMI" takes values {Normal, Overweight, Obese, Severely Obese}, each defined by a membership function.

Fuzzification: converting a crisp input ($BMI = 31.2$) into membership degrees across all its fuzzy sets.

Rule base: IF-THEN rules over linguistic variables. "IF BMI is Obese AND Age is Senior THEN Risk is High."

t-norm / t-conorm: the fuzzy generalisations of AND and OR. The usual choices are minimum for AND and maximum for OR.

Inference: evaluating every rule to get a firing strength, then aggregating their outputs. **Mamdani** inference produces a fuzzy output set; **Sugeno** produces a weighted numeric output and is easier to optimise.

Defuzzification: collapsing the aggregated fuzzy output back to one number, most commonly by taking the **centroid** (centre of gravity) of the output shape.

3.5.2 The design failure to avoid

TRAP — THE FUZZY LAYER AS A DECORATED LOOKUP TABLE

The default design in student projects and in much of the literature: the ANN outputs a probability, and the fuzzy system maps that single number to Low / Medium / High. This is a three-way threshold wearing elaborate mathematical costume. Any examiner will ask "why not just use two thresholds on the probability?" and there is no good answer, because there is no good answer — a single-input fuzzy system with a monotonic output is a smoothed threshold rule.

3.5.3 Two designs that actually justify their existence

Design A — fuzzy as a second, parallel information channel

The fuzzy inference system takes **multiple inputs**: the calibrated ANN probability *and* raw clinical variables. It can then encode clinical knowledge the ANN cannot express, and produce a human-readable rule trace.

```

Inputs:  P_ann    → {Low, Moderate, High}
        BMI      → {Normal, Overweight, Obese, Severely Obese}
        Age      → {Young, Middle, Senior}
        Modifiable → {Low, Medium, High} (composite, see Design B)

Output: Risk tier → {Minimal, Watch, Elevated, Priority Screening}

Example rules:
  IF P_ann is Moderate AND BMI is Obese AND Age is Senior
  THEN Risk is Priority Screening
  IF P_ann is High AND Modifiable is High
  THEN Risk is Priority Screening (and flag: intervention-responsive)
  IF P_ann is Moderate AND BMI is Normal AND Age is Young
  THEN Risk is Watch (de-escalate: model uncertain, no clinical corroboration)

```

The third rule is the point. The fuzzy layer can **disagree with the network** when clinical evidence is absent, and it can explain why. That is a capability the ANN alone does not have, and it is the answer to "why not just threshold the probability."

Design B — modifiable vs non-modifiable risk decomposition

This is where the project's claim to impact lives. Split the risk factors:

- **Non-modifiable:** Age, Sex, and family history proxies. Nothing the person can do.
- **Modifiable:** BMI, PhysActivity, Smoker, Fruits, Veggies, HvyAlcoholConsump, and to a degree HighBP and HighChol.

Run two fuzzy inferences and output two numbers: *baseline risk* from fixed factors, and *additional risk attributable to modifiable factors*. The system then reports not just "your risk is elevated" but "of your elevated risk, this portion is potentially addressable, and the largest single contributor is X."

WHY WE DIFFER

This converts a prediction into a **decision-support output**. It is what distinguishes a screening tool a person can act on from a number that makes them anxious. It is also genuinely novel on this dataset — the literature is almost entirely accuracy-chasing — and it is what makes the "noble and impactful" framing substantive rather than rhetorical.

3.5.4 Building the rule base

Three sources, used together, and the fact that you can articulate three sources is itself worth writing up:

1. **Clinical knowledge.** Established risk factors from diabetes screening guidelines (ADA risk-test logic and similar). Cite them.
2. **Rules extracted from Member 1's rough set analysis.** Reducts yield IF–THEN rules with measurable support and confidence. Convert the high-confidence ones into fuzzy rules. This is the concrete link that makes rough sets and fuzzy logic one system rather than two adjacent stages.
3. **Data-driven refinement.** Member 4 tunes membership function parameters against a validation objective.

Keep it to 15–30 rules. Beyond that the system stops being interpretable, which forfeits its only advantage over the ANN.

3.5.5 Implementation note

Options: `scikit-fuzzy` is the common choice but is sparsely maintained; `simpful` is a lighter alternative. **Recommendation: implement Mamdani inference directly in NumPy.** It is roughly 100–150 lines — membership functions, min/max t-norms, rule evaluation, aggregation, centroid defuzzification. You gain full control over the parameters Member 4 needs to optimise, you avoid dependency risk, and you can describe the exact inference procedure in the paper rather than citing a library's defaults. For a soft computing course, hand-implementing the inference engine is also worth marks.

COMPUTE

Machine A, entirely. Fuzzy inference on 38,000 validation rows with 25 rules is a handful of vectorised NumPy operations — well under a second. Even a genetic algorithm tuning MF parameters over 100 generations is minutes, not hours. Member 3 never needs Machine B.

3.5.6 Evaluating the fuzzy layer

You must show the fuzzy stage **adds something**. Report:

- Tier-wise observed diabetes prevalence — does prevalence rise monotonically across Minimal → Watch → Elevated → Priority? A bar chart of this is a strong, immediately legible figure.
- Screening efficiency: at a fixed budget (say, we can test 20% of the population), how many true cases does the fuzzy tiering capture versus simple probability thresholding?
- Agreement and, more interestingly, **disagreement** analysis: characterise the cases where the fuzzy tier departs from the raw probability ranking. Are those departures clinically sensible?
- Rule coverage: what fraction of cases fire at least one rule with strength above 0.5? Low coverage means gaps in the rule base.

3.6 Stage 5 — Optimisation and Integration M4

3.6.1 Three optimisation targets, in priority order

#	Target	Method and payoff
1	Multi-objective feature subset selection	NSGA-II minimising the number of features while maximising PR-AUC. Produces a Pareto front — the set of configurations where you cannot improve one objective without sacrificing the other. Overlay Member 1's rough set reducts on this front to show whether RST lands on, near, or off it. This is the single best figure in the paper.
2	Fuzzy membership function tuning	A genetic algorithm over the MF breakpoint parameters, optimising a clinically meaningful objective (e.g. cases detected within a fixed screening budget) rather than raw accuracy. Closes the loop between Stages 4 and 5 and gives Member 4 real ownership rather than glue work.
3	GA-based reduct search	Compare against Member 1's greedy QuickReduct. Directly addresses the "greedy finds a reduct, not the reduct" limitation from §3.3.5, and is a self-contained experiment with a clean result either way.

PLAIN ENGLISH — PARETO FRONT AND NSGA-II

With two competing objectives there is no single best answer. A solution is **Pareto-optimal** if no other solution is better on one objective without being worse on the other. The set of all such solutions is the **Pareto front**. NSGA-II is a genetic algorithm that evolves a whole population toward that front simultaneously rather than collapsing the objectives into one weighted score. Here it answers a question every reader has: "how much accuracy does each feature you drop actually cost?"

TRAP — NSGA-II WITH FULL MLP TRAINING IS UNAFFORDABLE

A population of 40 over 50 generations is 2,000 fitness evaluations. At two minutes per MLP training that is 67 hours of compute. It will not fit in your semester.

Mitigations, use all three: (i) evaluate fitness with a fast surrogate — LightGBM with a small tree count, or an MLP capped at 15 epochs — then retrain only the final Pareto-front members properly; (ii) cache fitness by feature-subset hash, since GAs re-evaluate identical genotypes constantly and caching typically saves 30–50% of evaluations; (iii) evaluate on a stratified 50,000-row subsample during search. Budget this properly in week one or it will not happen.

3.6.2 Integration deliverable

A single end-to-end script or small Streamlit application: user enters the reduct-selected answers → ANN produces a calibrated probability → fuzzy system produces a tier, a rule trace and a modifiable-risk breakdown. This is the demo for your presentation, and a live demo is worth considerably more in a viva than another table.

Member 4 also owns the shared repository, the environment specification, and the results registry — one append-only CSV where every experiment writes a row: timestamp, git commit, data hash, feature set name, model, hyperparameters, seed, and all metrics. When you write the paper in week nine, this file is your results section. Without it you will be re-running experiments to recover numbers you already computed.

COMPUTE

Stage 5 is the only genuinely compute-hungry part of the project, and the only place Machine B earns its keep. Everything else now runs comfortably on the laptop.

Even at Machine A's measured speed, a naive NSGA-II run — population 40, 50 generations, full MLP fitness — is 2,000 evaluations at roughly a minute each, so around 33 hours sequential or 16 with four workers. Still too long to run twice. **The mitigations in §3.6.1 remain mandatory**, not optional: surrogate fitness, hash caching, and subsampled evaluation. With all three the run drops to 2–4 hours on Machine A, or under an hour on Machine B.

Use `pymoo` for NSGA-II and DEAP or `pymoo` for the single-objective GAs. The fuzzy MF tuning and GA reduct search are cheap and belong on Machine A.

3.7 Stage 6 — Bias and equity audit Shared, led by M2

The payoff for the framing work in §1.2. Small in code, large in contribution.

1. **Subgroup performance.** Recompute PR-AUC, recall at your operating point, and calibration error separately within each level of Sex, Age band, Income band, `AnyHealthcare` and `NoDocbcCost`. Report as a table with confidence intervals from bootstrapping the test set.
2. **The specific hypothesis to test.** Recall is lower among respondents with `NoDocbcCost` = 1 and low `Income`, because undiagnosed true positives in those groups are labelled negative and therefore train the model to score them lower. State the hypothesis, then test it.
3. **Label-noise sensitivity analysis.** Simulate correcting the bias: take negatives in low-access subgroups whose feature profile is high-risk, flip a plausible fraction π to positive, retrain, and observe how subgroup metrics move as π varies from 0 to 0.25. This is a proper sensitivity analysis and it is the kind of thing that makes a reviewer take an undergraduate paper seriously.
4. **Discussion.** Connect it back: a screening tool intended to serve populations without lab access, trained on data where those same populations are systematically mislabelled, inherits an inverted bias. Naming the mechanism precisely is the contribution.

COMPUTE

Machine A throughout. Subgroup tables and bootstrap intervals take minutes. The sensitivity analysis retraines at each value of π — roughly 6 values $\times$ 5 seeds = 30 runs, about 15 minutes with four workers.

Part IV — What Makes This Different

If someone asks "what is new here?", this table is the answer. Keep it visible during the presentation.

Aspect	Typical approach on this dataset	Ours
Framing	"Early detection of diabetes"	Non-invasive risk stratification and screening triage. Honest about a cross-sectional, self-reported-label dataset.
Duplicates	Rarely mentioned; split performed on raw data	Measured, removed before splitting, with both versions reported so prior results remain comparable.
Leakage	All 21 features used without question	Consequence-suspect features (GenHlth, DiffWalk, PhysHlth) identified and the full experiment repeated without them.
Class balance	The 50/50 downsampled variant, so accuracy looks good	Full imbalanced data as primary; the balanced variant only as a comparability appendix.
Headline metric	Accuracy, occasionally F1	PR-AUC, recall at a fixed clinical operating point, and calibration. Accuracy shown only beside the majority-class baseline.
Calibration	Almost never reported	Brier score and reliability diagrams throughout — structurally required, since the fuzzy layer consumes probabilities.
Rough sets	Classical Pawlak RST applied directly, one reduct reported, no discussion of whether it was appropriate	Both failure modes demonstrated numerically; VPRS β -sensitivity curve; a prevalence-relative variant for imbalance; fuzzy-rough alternative; bootstrap stability analysis.
Feature selection comparison	Rough sets alone, no competing methods	Benchmarked against mutual information, LASSO, RFE and an NSGA-II Pareto front at matched feature counts.
Imbalance handling	SMOTE, applied without comment, sometimes before the split	Four-way ablation, SMOTE-NC rather than vanilla SMOTE, with the calibration cost of resampling measured.
Fuzzy layer	Bins the ANN probability into three tiers	Multi-input inference combining probability with clinical variables; produces rule traces and a modifiable/non-modifiable risk decomposition. Rules partly derived from the rough set stage.
Statistics	Single run, single seed, no variance	Five seeds minimum, mean $\pm$ SD, McNemar / 5×2 cv significance testing.
Equity	Not addressed	Full subgroup audit plus a label-noise sensitivity analysis grounded in a named, plausible mechanism.

You do not need every row to succeed. Landing five or six of them well produces a project that is clearly above coursework level. Attempting all twelve badly produces one that is not.

Part V — Experimental Protocol

5.1 The main results table

This is the table the whole project exists to fill. Every cell is mean $\pm$ SD over five seeds. Prevalence baseline for PR-AUC is roughly 0.14 — print it in the caption.

Feature set	k	Model	PR-AUC	ROC-AUC	Rec@P=.30	Brier
All features	21	LR / GBM / MLP				
Classical RST reduct	?	"				
VPRS reduct (best β)	?	"				
Prevalence-relative VPRS	?	"				
Fuzzy-rough reduct	?	"				
GA-searched reduct	?	"				
Mutual information top-k	=k above	"				
LASSO selection	?	"				
Recursive feature elimination	=k	"				
Leakage-free subset	18	"				

COMPARISON DISCIPLINE

Compare feature selection methods **at matched feature counts**. If the rough set reduct has 9 features, compare it against the top 9 by mutual information, not against LASSO's arbitrary 14. Otherwise you are measuring dimensionality, not selection quality, and the comparison says nothing.

5.2 Supporting tables and figures

1. **β -sensitivity.** Reduct size, γ_β , and downstream PR-AUC against $\beta \in [0.50, 1.00]$. Three lines, twin y-axes.
2. **Discretisation comparison.** Clinical vs equal-width vs supervised cutpoints $\rightarrow \gamma$, reduct size, PR-AUC.
3. **Attribute stability.** Selection frequency of each of the 21 attributes across 50 bootstrap resamples, as a horizontal bar chart.
4. **Imbalance ablation.** Four strategies $\times$ PR-AUC and Brier, showing the accuracy/calibration trade-off.
5. **Reliability diagram.** Before and after calibration.
6. **Pareto front.** Feature count vs PR-AUC from NSGA-II, with rough set reducts overlaid as labelled points.
7. **Fuzzy tier validity.** Observed diabetes prevalence within each output risk tier.
8. **Subgroup performance.** Recall and calibration error by Income, AnyHealthcare, NoDocbcCost, with bootstrap CIs.
9. **Label-noise sensitivity.** Subgroup recall as a function of the assumed undiagnosed fraction π .

Nine figures is roughly right for a paper of this scope. Figures 1, 6 and 8 are the ones that carry the argument; if time is short, make those three excellent and the rest adequate.

Part VI — Compute Plan

The figures in this part are measured on Machine A, not estimated. Re-run the benchmark in Appendix A if you change hardware.

6.1 The two machines

Machine A	Lenovo, Intel Core Ultra 5 125H, Arc integrated GPU, 16 GB RAM. Meteor Lake hybrid architecture: 4 performance cores, 8 efficiency cores and 2 low-power efficiency cores, 18 logical threads in total. The Arc iGPU has no dedicated memory — it carves its allocation out of the same 16 GB the operating system and Python are using.
Machine B	NVIDIA RTX 5050 (Blackwell, 8 GB GDDR7) with a 14th-generation Intel HX CPU. Optional for this project. See §6.5 for the one job where it helps.

THE HEADLINE FINDING

Machine A runs this entire project on CPU alone, in a few hours of total wall-clock time. That is a measurement, not an estimate. A full 100-epoch MLP training takes about 55 seconds. The largest planned sweep — 160 runs — takes roughly 75 minutes with four parallel workers.

Nothing here is an overnight job. Machine B is genuinely optional and worth using for exactly one task (§6.5). Do not build the project plan around waiting for hardware.

6.2 Why CPU, and why not the Arc iGPU

The Arc iGPU is technically usable — Intel Extension for PyTorch exposes an `xpu` backend that works on Meteor Lake. It is nonetheless the wrong choice here, for three reasons:

- **It has no memory of its own.** Any allocation comes out of the same 16 GB you are already budgeting carefully. You would be spending scarce RAM to accelerate a model that trains in under a minute on the CPU.
- **The ecosystem is thin.** IPEX version compatibility with PyTorch is fussy, and when something breaks there is little community material to work from.
- **The model is too small to benefit.** At roughly 3,500 parameters, per-step cost is dominated by framework overhead rather than arithmetic — precisely the regime where an integrated GPU performs worst.

The one place the Arc is worth touching is **OpenVINO for inference** in the Streamlit demo. Low risk, and it fits the "deployable on modest hardware" framing. Strictly a week-8 nice-to-have.

Install the CPU-only PyTorch build (Appendix A). It is roughly 200 MB rather than 2.5 GB, and it removes an entire class of CUDA-related failure from the project.

6.3 The threading result — and why more threads is worse

Benchmark output from Machine A, one process, 253,680 rows, batch 1024:

PyTorch threads	sec / epoch
4	0.541 ← best
6	0.693
8	0.633
10	0.910
14	0.786

PLAIN ENGLISH — WHY 4 BEATS 14 ON A HYBRID CPU

The 125H has three kinds of core running at different speeds: 4 fast P-cores, 8 slower E-cores, and 2 slower still low-power E-cores. Neural network training synchronises at every gradient step — roughly 248 times per epoch — and **each step can only finish when its slowest thread finishes**. Once you request more than 4 threads, work lands on E-cores and every single step waits for them. You are not adding capacity; you are adding a slower critical path.

The non-monotonic wobble (6 slower than 8, 10 slower than 14) is Windows scheduler noise. It does not reliably pin threads to core types, so individual measurements jitter. Read the trend, not the wobble.

6.4 The parallelism strategy: narrow workers, several at once

Because one training run cannot saturate the machine, the correct strategy is the opposite of the GPU case. Instead of one wide job, run several narrow ones concurrently. Measured, four processes at four threads each:

Configuration	sec / epoch, each	Effective throughput
1 process × 4 threads	0.541	1.85 epochs/sec
4 processes × 4 threads	1.01	3.96 epochs/sec — 2.1×
4 processes × 14 threads	2.91	1.37 epochs/sec — worse than serial

Four workers slow each other by 1.86×, not 4×, so net throughput roughly doubles. It is not the ideal 4× because you are requesting 16 threads from a machine with 4 fast cores, and because memory bandwidth is shared. The bottom row is the cautionary case: oversubscribing both dimensions at once is *slower than running the jobs one at a time*.

TRAP — OVERSUBSCRIPTION COMPOUNDS

It is tempting to reason "18 threads available, 4 workers, so 4 threads each is wasteful — give them more." That reasoning produces the bottom row of the table above, a 2.9× slowdown per process and worse aggregate throughput than a single serial run. When workers multiply threads, the product must stay near the **P-core count**, not the logical thread count.

Resulting runtimes for the whole project

Job	Serial	4 workers
Single MLP, 100 epochs	~55 s	—
Main grid: 160 runs	~2.5 h	~75 min
Optuna, 60 trials	~55 min	~25 min
LightGBM tuning, 100 trials	~1.5 h	20–40 min
Bootstrap reduct stability, 50×	~2 h	~30 min
FRFS, 15 subsamples of 6,000	~1 h	—
Label-noise sensitivity, 30 runs	~30 min	~15 min
NSGA-II with §3.6.1 mitigations	—	2–4 h

How to implement it

```
from config import N_WORKERS          # see Appendix A
from joblib import Parallel, delayed

results = Parallel(n_jobs=N_WORKERS)(
    delayed(train_one)(cfg) for cfg in configs
)
```

For Optuna specifically, prefer launching **separate processes** against a shared SQLite study rather than the `n_jobs` argument. Separate processes sidestep Python's global interpreter lock entirely and isolate crashes:

```
study = optuna.create_study(storage="sqlite:///study.db",
                           study_name="mlp", load_if_exists=True,
                           direction="maximize")
study.optimize(objective, n_trials=20)      # run in 4 terminals
```

6.5 When to use Machine B

One job: **NSGA-II**. Even with the surrogate-fitness mitigations it is 2–4 hours on Machine A, and it is the one task you may want to run more than once as you tune the objective. On the RTX it drops to under an hour.

Everything else stays on Machine A. The coordination overhead of shuttling code, data and results between two machines is real, and for a job that finishes in 30 minutes it is not worth paying. If Machine B is inconvenient to access, **drop it entirely** — the project completes without it.

IF YOU DO USE MACHINE B — THE BLACKWELL CUDA TRAP

The RTX 50-series uses NVIDIA's Blackwell architecture, whose compute capability is newer than the 40-series. PyTorch wheels built against older CUDA toolkits contain no compiled kernels for it. An older `torch` build fails at the first GPU operation with:

```
CUDA error: no kernel image is available for execution on the device
```

This is confusing because `torch.cuda.is_available()` returns `True` and the device name prints correctly. It only fails when a kernel actually launches. **Fix:** install a build targeting CUDA 12.8 or newer, taking the current command from pytorch.org rather than copying a version string from any document, including this one. Verify with a real operation:

```
import torch
print(torch.__version__, torch.version.cuda)
print(torch.cuda.get_device_capability(0))
x = torch.randn(1000, 1000, device='cuda')
print((x @ x).sum().item()) # this line is the real test
```

6.6 Memory discipline on Machine A

16 GB shared with an integrated GPU is now the project's only real constraint. Compute is not tight; memory is.

- **Cast to float32 on load.** Pandas defaults to float64, doubling memory for no benefit. Use `int8` for the fourteen binary columns — they are otherwise stored as 8-byte floats, an eightfold waste.
- **Never materialise anything $n \times n$.** The one hard rule. $253,680^2$ as float32 is roughly 250 GB. Every pairwise operation must be subsampled or chunked — this is why FRFS runs at 6,000 objects (§3.3.4).
- **Prefer scripts over notebooks for long jobs.** A script releases everything on exit. Jupyter retains references to every output cell, so a notebook left running through a sweep will slowly consume your RAM.
- **Delete and collect explicitly** in long sessions: `del big_object; import gc; gc.collect()`.
- **Watch the worker multiplier.** Four parallel workers means four copies of the resident dataset. At 20 MB each that is trivial here, but it stops being trivial if you ever cache per-worker intermediates.

6.7 Keeping machines and members in sync

- **One Git repository.** Code, the split index file, reduct JSONs, the results registry CSV, and the environment specification. Raw data does not go in it — add to `.gitignore` and share the download link.
- **Assert the data hash** at the top of every script. This catches the failure where someone re-downloaded the CSV or skipped deduplication and has been producing results on different data for a week.
- **One append-only results CSV.** Every run writes a row: timestamp, machine, git commit, data hash, feature set, model, hyperparameters, seed, all metrics. Never overwrite. Resolve merge conflicts by keeping both rows.
- **Pin the environment.** Commit `requirements.txt` with exact versions. If anyone uses Machine B they will need a different PyTorch build — note that in the README so nobody "fixes" it.

Part VII — Software Stack

Purpose	Library	Notes
Data handling	pandas, numpy	Cast to float32 / int8 on load.
Classical ML, metrics	scikit-learn	<code>average_precision_score</code> , <code>brier_score_loss</code> , <code>CalibratedClassifierCV</code> , <code>StratifiedKFold</code> .
Gradient boosting	LightGBM or XGBoost	The realistic strong baseline. Do not skip it.
Neural network	PyTorch	See the CUDA note in §6.4. Keras is acceptable if the team prefers it, but PyTorch integrates more easily with Member 4's optimisation loops.
Rough sets	Write it yourself	No maintained Python RST library is dependable. The groupby implementation in §3.3.5 is about fifty lines and is exactly right for this scale. Writing it is also worth marks.
Fuzzy-rough	fuzzy-rough-learn, or self-implemented	Verify the library's behaviour on your toy example before trusting it on real data.
Fuzzy inference	Write it yourself (NumPy)	scikit-fuzzy is sparsely maintained; simplful is lighter. Hand-implementing Mamdani gives Member 4 the parameter access needed for MF tuning.
Imbalance handling	imbalanced-learn	Use <code>SMOTENC</code> , not <code>SMOTE</code> , and always inside an <code>imblearn.Pipeline</code> .
Hyperparameter search	Optuna	Far more sample-efficient than grid search, and it persists studies to disk so an interrupted run resumes.
Multi-objective GA	pymoo	NSGA-II implementation. DEAP is an alternative for single-objective work.
Statistics	scipy, statsmodels	McNemar's test, bootstrap confidence intervals.
Plotting	matplotlib, seaborn	Save at 300 dpi to <code>figures/</code> as you go.
Demo	Streamlit	Fastest route to a live demo for the viva.

Part VIII — Timeline

Ten weeks, assuming part-time work alongside other courses. A compressed fallback follows.

Week	Shared / M1	M2	M3 / M4
1	Stage 0 together. Verify all §2.2 properties. Freeze the test set. Repo, environment, results-CSV format agreed.	Environment setup per Appendix A, including the threading benchmark. Confirm sec/epoch before planning anything.	M4 sets up the repo and results registry. M3 reads fuzzy theory.
2	EDA. Discretisation scheme decided jointly with M3 . Reduct JSON format agreed with M2.	Baselines: majority, LR, tree, LightGBM on all 21 features. This unblocks everyone.	M3 drafts membership functions using the agreed bins. M4 scopes NSGA-II compute budget.
3	Classical RST. Demonstrate both failure modes numerically. This is a result, not a setback.	MLP implemented and training end to end. Imbalance ablation started.	M3 builds Mamdani engine in NumPy. M4 implements GA reduct search.
4	VPRS β sweep. Prevalence-relative variant.	Imbalance ablation complete. Calibration and reliability diagrams.	M3 drafts rule base v1 from clinical sources.
5	Bootstrap stability on Machine B. FRFS on subsamples.	All feature sets $\times$ all models $\times$ 5 seeds. The main table starts filling.	M3 integrates M1's extracted rules. M4 runs first NSGA-II.
6	Reduct set finalised and delivered. Rule extraction for M3.	Optuna tuning. Significance testing.	M3 evaluates tier validity. M4 tunes MFs.
7	Integration week. End-to-end pipeline runs. Streamlit demo working. Stage 6 equity audit executed. Everyone stops adding features.		
8	Final test-set evaluation — the single permitted touch. All figures produced at publication quality. Paper drafting begins in parallel.		
9	Paper written from the results registry. Slides built. Demo rehearsed.		
10	Buffer. Something will have gone wrong; this is where you fix it. Do not plan work here.		

A NOTE ON PACING

Because the measured runtimes in Part VI are short, no week in this timeline is gated on compute. The critical path is **decisions and integration**, not training time. If a week slips it will be because an interface was not agreed or a result was ambiguous, not because a job was still running. Budget your attention accordingly.

Compressed six-week fallback

If the semester is shorter than expected, cut in this order: **first** the fuzzy-rough branch (keep VPRS), **second** the GA reduct search, **third** the label-noise sensitivity simulation (keep the descriptive subgroup audit, which is cheap). **Never cut:** deduplication, the frozen test set, multiple seeds, PR-AUC as headline metric, or the leakage-free comparison. Those five are what make the results trustworthy; everything else is scope.

Part IX — Risk Register

Risk	Likelihood	Mitigation
Rough set reducts perform <i>worse</i> than mutual information	Moderate	Report it honestly. A negative result with a stability analysis and a β curve explaining <i>why</i> is a legitimate paper. Do not fudge. Examiners can tell.
Gradient boosting beats the MLP	High	Expected on tabular data. Report it, and state the MLP's actual role per §3.4.1.
γ degenerates and no reduct is found	High	Anticipated in §3.3.2. This is the motivation for VPRS, not a failure. Have the numbers ready.
NSGA-II does not finish in time	Moderate	Surrogate fitness, caching, subsampling — all three, budgeted in week one. Fallback: a coarse grid over feature counts still produces a usable front.
PyTorch will not run on the RTX 5050	Moderate	§6.4. Resolve in week one. Machine A can run everything if needed.
Members' pieces do not integrate	High	The two interface contracts (§3.3.7 and §3.4.6) are the specific defence. Agree both in week two, in writing, with a dummy file each side can test against.
Results lost or unreproducible at write-up	High	Append-only results registry with git commit and data hash on every row. Non-negotiable.
Test set contaminated by repeated evaluation	Moderate	Load the test split in exactly one script, run in week eight. Put the rule in the README.
One member falls behind	Moderate	The dependency chain is M1→M2→M3. Mitigate by having M2 start on all 21 features immediately, and M3 build against a dummy probability file, so nobody is idle waiting.

Part X — Paper and Presentation

Suggested paper structure

1. **Introduction** — diabetes burden, the undiagnosed fraction, why non-invasive screening triage matters in low-resource settings. Contributions listed as bullets.
2. **Related work** — ML on BRFSS diabetes data; rough sets in medical feature selection; fuzzy systems in clinical decision support. Note explicitly what prior work on this dataset does not report (calibration, duplicates, subgroup performance).
3. **Data** — BRFSS provenance, the feature dictionary, and a frank statement of the cross-sectional and self-report limitations up front, not buried.
4. **Methodology** — one subsection per stage. The rough set subsection carries the theoretical weight: state both failure modes, then motivate VPRS and the prevalence-relative extension from them.
5. **Experimental setup** — splits, seeds, metrics, hardware, significance tests.
6. **Results** — the main table and the nine figures.
7. **Bias and equity analysis** — its own section, not a subsection. This is a distinguishing contribution and should be visible in the table of contents.
8. **Discussion and limitations** — what the system can and cannot claim; what a longitudinal dataset would enable.
9. **Conclusion and future work.**

Presentation guidance

- **Lead with the reframing.** Slide two should be "why the obvious version of this project is wrong." It signals seriousness immediately and pre-empts the most likely question.
- **Show the accuracy trap live.** One slide: a model with 86% accuracy that detects zero diabetics. It lands every time and it justifies your entire metric choice in ten seconds.
- **Run the demo.** A live Streamlit walkthrough — enter answers, see the probability, see the tier, see the rule trace and the modifiable-risk breakdown — is worth more than three results slides.
- **End on the equity finding.** It is the most memorable thing you have and it closes the loop with the "noble and impactful" framing you opened with.

Appendix A — Environment Setup

Every step below was hit in practice during setup. Following it in order avoids roughly two hours of debugging.

A.1 Create the project and virtual environment

Work in a dedicated folder, **not** your Windows user directory. Creating the venv in `C:\Users\you` works but scatters project files among your Documents and Downloads.

```
mkdir diabetes-project
cd diabetes-project
python -m venv .venv
.\.venv\Scripts\Activate.ps1
python -m pip install --upgrade pip
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install numpy pandas scikit-learn matplotlib seaborn lightgbm optuna joblib
```

The `cpu` index URL matters: it pulls a roughly 200 MB wheel rather than a 2.5 GB CUDA build you would never use on Machine A.

If PowerShell blocks activation with an execution-policy error, run once:

```
Set-ExecutionPolicy -Scope CurrentUser RemoteSigned
```

A.2 Point VS Code at the right interpreter

File → Open Folder on the project directory. Trust the folder when the Restricted Mode banner appears — until you do, the Run button and interpreter selection misbehave. Then `Ctrl+Shift+P` → **Python: Select Interpreter** → choose the one under `.venv\Scripts`.

TRAP — THE PROMPT SAYS (.VENV) BUT SYSTEM PYTHON RUNS

The most confusing early failure: your terminal shows the `(.venv)` prefix, yet `import torch` fails because the command is resolving to `AppData\Local\Programs\Python\python.exe`. Always verify explicitly rather than trusting the prompt:

```
Get-Command python | Select-Object Source
python -c "import torch; print(torch.__version__)"
```

The first must print a path ending in `diabetes-project\.venv\Scripts\python.exe`. If it does not, reopen the terminal after selecting the interpreter.

Note separately that `.venv\pyvenv.cfg` pointing at your system Python is **correct and expected** — a venv reuses the base interpreter by design. That file is not evidence of this bug.

A.3 Pasting long scripts

TRAP — SILENT TRUNCATION

Pasting a multi-line script into the VS Code terminal truncates it: PowerShell treats a blank line as "submit" and discards everything after. Pasting into an editor tab that already contains content can also merge badly.

Symptom: the script runs, prints nothing at all, and exits with code 0. No traceback, because the file simply ends early.

Always verify after writing a file:

```
Get-Content script.py | Measure-Object -Line
```

If the count is wrong, clear the editor fully with `Ctrl+A` then `Delete` before re-pasting, or use `notepad script.py` — Notepad has no autocomplete or formatter to mangle the paste. This problem recurs as scripts grow; the durable fix is committing to Git early and often so a known-good copy always exists.

A.4 The threading benchmark

Run this before planning any sweeps. It takes two to four minutes. Plug the laptop in and set Windows power mode to **Best Performance** first, or you will be measuring the power profile rather than the CPU.

```

import os
os.environ["OMP_NUM_THREADS"] = "8"
os.environ["MKL_NUM_THREADS"] = "8"
import time, torch, torch.nn as nn
torch.manual_seed(0)
N, D, BATCH, EPOCHS = 253_680, 21, 1024, 5
X = torch.randn(N, D)
y = (torch.rand(N) < 0.14).float()
def make_model():
    return nn.Sequential(nn.Linear(D, 64), nn.BatchNorm1d(64), nn.ReLU(), nn.Dropout(0.3),
                        nn.Linear(64, 32), nn.BatchNorm1d(32), nn.ReLU(), nn.Dropout(0.2),
                        nn.Linear(32, 1))
def run_epoch(model, opt, crit):
    perm = torch.randperm(N)
    for i in range(0, N, BATCH):
        idx = perm[i:i + BATCH]
        opt.zero_grad()
        crit(model(X[idx]).squeeze(1), y[idx]).backward()
        opt.step()
print("logical cores:", os.cpu_count(), flush=True)
for threads in [4, 6, 8, 10, 14]:
    torch.set_num_threads(threads)
    model = make_model()
    opt = torch.optim.AdamW(model.parameters(), lr=1e-3)
    crit = nn.BCEWithLogitsLoss(pos_weight=torch.tensor(6.0))
    run_epoch(model, opt, crit)
    t0 = time.perf_counter()
    for _ in range(EPOCHS):
        run_epoch(model, opt, crit)
    print(f"threads={threads:2d} {(time.perf_counter()-t0)/EPOCHS:.3f} sec/epoch", flush=True)

```

Two details matter. The `os.environ` lines must precede `import torch`, because the OpenMP and MKL thread pools initialise at import and cannot be changed afterwards. And the untimed warm-up epoch is discarded deliberately: the first pass pays one-time allocation and thread-pool startup costs that would otherwise bias the first configuration measured.

Run with `python bench.py`, or **Ctrl+F5** in VS Code. Do not use **F5** — the debugger attaches a tracer that distorts the timings.

A.5 Lock in the configuration

Put the result in one file and import it first in every script, before `torch`:

```

# config.py
import os
N_WORKERS = 4
THREADS_PER_WORKER = 4
os.environ["OMP_NUM_THREADS"] = str(THREADS_PER_WORKER)
os.environ["MKL_NUM_THREADS"] = str(THREADS_PER_WORKER)

```

```

# any training script
from config import N_WORKERS
import torch
torch.set_num_threads(4)

```

KNOW WHEN TO STOP TUNING

There is a rabbit hole here — thread affinity masks, `KMP_AFFINITY`, disabling E-cores in BIOS. Each might buy 10–20%. Given that the project's total compute is a few hours, that saves perhaps half an hour across an entire semester, and you will spend longer configuring it than you recover. Take the benchmark result, set two numbers, and move on.

A.6 Repository hygiene

```

git init
"__pycache__\`n.venv\`n*.csv\`nfigures/" | Out-File -Encoding utf8 .gitignore
pip freeze > requirements.txt
git add -A; git commit -m "environment setup + benchmark"

```

Commit at this point, before any project code exists. It gives you a clean baseline and a known-good environment specification to hand to the other three members.

Appendix B — First 48 Hours

Do these before anything else. Together, in one sitting if possible.

Action	
☐	Download the dataset. Confirm 253,680 rows and 22 columns.
☐	Run every check in §2.2. Write the numbers into a shared document. Several design decisions depend on them.
☐	Count contradictory groups (same features, different label). This number tells you in advance how badly classical RST will degenerate.
☐	Deduplicate, split 70/15/15 stratified, save <code>splits.npz</code> , record the data hash, commit it.
☐	Create the Git repo. Add <code>.gitignore</code> for raw data. Create the empty results-registry CSV with agreed column headers.
☐	On Machine B: install PyTorch and run an actual CUDA matmul per §6.4. Do not assume it works because the device is detected.
☐	Implement <code>gamma_vprs</code> from §3.3.5 and verify it against the six-person toy example (expect $\gamma = 0.667$ at $\beta = 1$).
☐	Train a logistic regression on all 21 features. Record PR-AUC. This is the number every later result must beat.
☐	Compute the majority-class accuracy and write it on a slide now, while it is fresh.
☐	Agree both interface contracts (§3.3.7 reduct JSON, §3.4.6 probability CSV) in writing. Create a dummy file for each so both sides can develop independently from day one.

A CLOSING NOTE ON SCOPE

This document deliberately describes more than four students can execute in one semester. That is intentional — it is easier to cut from an over-specified plan than to invent additions under deadline pressure. Use Part IV to decide what to keep: five or six strong differentiators, executed carefully with honest statistics, beats twelve attempted superficially. The parts that are genuinely non-negotiable are the five listed at the end of Part VIII.

Appendix C — Decision Log

Fill this in as you go and keep it in the repository. When you write the paper in week nine you will need these numbers and will not remember where they came from. Values below marked **measured** are already known; the rest are yours to fill.

Item	Value	Notes / where it came from
Rows, raw		Expect 253,680
Exact duplicate rows		<code>df.duplicated().sum()</code>
Feature-identical rows		Ignoring the label
Contradictory groups		Drives Stage 2 — predicts how badly classical RST degenerates
Rows after dedup		
Class prevalence		Also the PR-AUC baseline
Data hash		Asserted by every script
Split seed / proportions	42 / 70-15-15	Stratified
Discretisation chosen		Clinical / equal-width / supervised — and why
γ without discretisation		Failure mode A evidence
γ with discretisation, $\beta=1$		Failure mode B evidence
Chosen β		From the sensitivity curve
Reduct size at chosen β		
LR baseline PR-AUC		The number everything must beat
Majority-class accuracy		For the accuracy-trap slide
Clinical operating point		e.g. precision = 0.30
sec/epoch, 4 threads	0.541 (measured)	Machine A, Ultra 5 125H, batch 1024
Worker configuration	4 × 4 threads (measured)	1.01 s/epoch each; 2.1× net throughput
torch version	2.14.0+cpu (measured)	CPU build, no CUDA
Final test evaluation date		Should occur exactly once

Appendix D — Glossary

Term	Meaning
BRFSS	Behavioral Risk Factor Surveillance System — the annual US telephone health survey this data comes from.
Cross-sectional	All variables measured at one point in time, with no follow-up. Contrast with longitudinal.
Boundary region	In rough set theory, objects that cannot be classified with certainty using the chosen attributes.
Brier score	Mean squared error of predicted probabilities; a measure of calibration. Lower is better.
Calibration	Whether predicted probabilities match observed frequencies. Among people given 30% risk, do ~30% have the disease?
Core (RST)	Intersection of all reducts — the indispensable attributes.
Defuzzification	Converting a fuzzy output set back into a single number, usually by centroid.
Dependency degree (γ)	Fraction of objects classifiable with certainty using a given attribute subset.
Discretisation	Converting a numeric variable into labelled bins.
Equivalence class / granule	Set of objects with identical values on the chosen attributes.
Fuzzification	Converting a crisp value into degrees of membership across fuzzy sets.
Indiscernibility	The relation "identical on attribute subset B", which generates the granules.
Leakage	Information reaching the model that would not be available at real prediction time, inflating measured performance.
Lower approximation	Union of granules entirely contained in a target set — certain members.
Mamdani inference	Fuzzy inference producing a fuzzy output set, then defuzzified. Interpretable; contrast Sugeno.
Membership function	Maps a value to a degree of membership in $[0,1]$.
NSGA-II	Multi-objective genetic algorithm producing a Pareto front rather than one solution.
Pareto front	Solutions where no objective can improve without another worsening.
Positive region	Union of lower approximations across all decision classes.
PR-AUC	Area under the precision–recall curve. Primary metric under class imbalance; baseline equals prevalence.
Precision	Of those flagged positive, the fraction truly positive.
Recall / sensitivity	Of those truly positive, the fraction flagged.
Reduct	Minimal attribute subset preserving the dependency degree of the full set.
ROC-AUC	Probability a random positive scores above a random negative. Optimistic under heavy imbalance.
SMOTE / SMOTE-NC	Synthetic minority oversampling; the NC variant handles categorical features correctly.
Stratified split	Splitting so each partition preserves the overall class proportions.
t-norm / t-conorm	Fuzzy generalisations of AND and OR, typically minimum and maximum.
VPRS	Variable Precision Rough Sets — admits granules that are at least β pure, tolerating noise.